

What is it ?

- Algorithm used when our data set is ordered → we can make strategic jumps instead of traverse linear
- In binary search, it halves the search space in each iteration by checking the middle element and determining whether to search the left or right half of the remaining array

What is its complexity ?

- $O(\log n)$. Why ?
- We halve the size of N until we reach 1 (worst case), now we have $N/2^k = 1$ when
 - k : number of time that we halve the input size
- So do $\log(2)$ both side, we can have $\log(n) = k$. It also can say that k is the worst number of operation we do till we find the result → Boom ! Time complexity = $O(\log n)$ (ಠ_ಠ)-)

How does it work ?

- Let's play with some pseudo code first

```
BinarySearch(arr, lo, hi, n):
    do:
        m = ( lo + (high - low)/2 ) - Floor it
        v = arr[m]

        if n > v: - It must be in the right
            lo = m + 1
        else if n < v: - Hmm, It must be in the left now
            hi = m // we always have [lo, hi) - lo inclusive, hi exclusive
        else: - Yay, we found it ♪( ͡° ͜ʖ ͡° )
            return true
    while lo < hi

    return false - Nah, the thing you are finding just not here ε('ㅏㅑ')3
```